PROJECT HORIZON

Strategy & Execution Plan

The primary initiative is software for analysing the precision, errors, recovery and consequences of autonomous AI-agent tasks. A study agent is our first application and test environment—not the main invention.

Executive proposal | Revised October 2026 | Version 2.0

Prepared for potential collaborators, mentors, technical reviewers and prospective partners. Status: concept and feasibility research. No deployed product, tested advantage, users, revenue or proprietary algorithm is asserted.

1. Executive summary — the actual project thesis

Project Horizon aims to develop a reusable AI-agent reliability intelligence layer. It would observe multi-step agent executions, record deviations from defensible reference expectations, estimate their severity and likelihood of recovery, compare the final output with independently verified outcomes, and—if experiments support it—recommend or trigger verification only when its expected value exceeds its cost.

This is not a proposal to create a generic chatbot tutor and call its logs “reliability research.” The reliability engine is the intended core software. A small student-facing source-grounded research agent, provisionally called Horizon Study, is our first controlled operating environment. It has independent potential as a public product, but its initial strategic function is to produce real and measurable multi-step tasks in which we can study agent behaviour.

Component	Role	Who might ultimately use it
Horizon Reliability	Primary research and potential business: trace ingestion, error metrics, recovery and impact estimates, selective verification, final-outcome checks	Teams developing or deploying tool-using AI agents; researchers
Horizon Study	A functional education/research agent and first experimentation sandbox; potentially publishable independently	Students, independent learners, pilot testers

Core decision requested

Approve a bounded 90-day feasibility program that builds one small agent and one framework-neutral reliability layer in parallel, measures performance against credible baselines, and tests transfer to a second agent domain before making strong commercial claims.

What success would mean

A controlled result: the proposed trajectory-aware policy preserves a predefined level of final-outcome quality while reducing evaluation cost or costly missed failures relative to final-only, full-step and fixed-threshold baselines. User demand, product usability and transferability are separate, equally necessary tests.

2. Problem and conceptual approach

2.1 Why raw logs and final correctness are not enough

AI agents complete tasks by repeatedly interpreting a goal, selecting actions, calling tools, receiving observations, and deciding what to do next. Evaluation systems can already trace these steps and score outcomes. Yet a failure can originate in the tool, the model's decision, an incorrect interpretation of a tool response, or an unreliable verification rule. Seeing one incorrect action does not prove the final task failed; seeing a plausible final response does not prove a real-world effect occurred.

Two executions may have identical final scores while exhibiting very different risk profiles: one performs flawlessly; the other makes harmful intermediate choices but eventually recovers. Conversely, a seemingly tidy trace can mask a failed downstream database update. These distinctions motivate longitudinal evaluation, not merely a binary answer score.

2.2 The proposed five linked measurements

Measurement	Definition	Evidence needed
Location	First observable step at which behaviour materially deviates	Timestamped actions, observable state, reference criteria
Severity	Degree to which the deviation matters for correctness, safety or task objectives	Rubric or deterministic reference with calibrated uncertainty
Persistence	Whether the deviation propagates, compounds or is contained	Later states/outputs and dependency links
Recovery	Probability or observed fact of correction before termination	Repeated runs, checkpoints, labels and outcomes
Consequence	Effect of deviation on actual final result or environment state	Post-state verifier; controlled counterfactual replay when possible

2.3 Two complementary datasets

Dataset A — WHERE / WHEN: run identifier, step number, timestamp, action type, inputs and tool results, detected error onset, affected downstream steps, evidence behind each label and recovery events.

Dataset B — HOW MUCH / HOW FAR: rubric scores or observed state differences, deviation severity, estimated outcome risk and confidence, verified final outcome, the estimated cost of intervention, and counterfactual impact when measurable.

The dataset pair should preserve raw evidence and evaluator provenance. A second AI model is an evaluator—not an authoritative ground-truth source. For subjective tasks, multiple expert raters and disagreement measurements are preferable to invented binary certainty.

3. Product architecture and technology strategy

3.1 Intended data flow

Supported agent → event/trace adapter → normalized trajectory store → reference/evidence service → error and recovery evaluator → risk-based verification policy → independent final-state verifier → metrics and human-readable reliability report.

Layer	Design intent	Initial implementation
Agent and tools	Agent makes bounded, observable decisions with approved tools	Small Python ReAct-style controller; replaceable LLM adapter
Instrumentation	Capture model/tool calls, observations, timestamps, state changes	JSONL / SQLite; standard event schema
Reference engine	Use authoritative criteria and known-answer tasks	Deterministic tests and source passages initially
Evaluation	Find degradation, assess severity and uncertainty	Transparent rules first; optional calibrated LLM evaluator
Verification	Verify completed task outcome independently	Database read-back, source citation checks, artifact tests
Decision policy	Decide whether verification is cost-justified	Compare simple threshold to predictive policy
Reporting	Expose location, severity, recovery, effect and costs	Offline run report, then developer UI

3.2 NVIDIA as infrastructure, not our rival by default

NVIDIA NeMo Agent Toolkit provides useful agent workflow instrumentation and evaluation components, including support for extensions. We should evaluate whether those tools reduce the cost of collecting traces and executing our custom evaluations. Our potential differentiation is not owning a log recorder or claiming to have invented step scoring. It must be in validated methodology, evidence provenance, recovery/impact predictions and cost-aware intervention.

The central trace schema should remain provider-neutral, and the reliability engine should accept traces from at least one second agent workflow in later experiments. This protects against mistaking NeMo-specific setup for a general approach.

3.3 What we intentionally will not build first

A foundation language model, full agent marketplace, autonomous external purchases, broad browser automation, high-stakes professional advice or production personal-data infrastructure. These add costs and risks without first answering our scientific question.

4. Horizon Study — the first agent and optional product

4.1 Why education is an effective testbed

A useful study agent can operate with source documents and explicitly graded tasks, making it easier to observe wrong citations, unsupported claims, incorrect extraction, premature completion and correction. We can also use it personally, show it to real students and later publish a small application. However, student feedback cannot alone validate agent reliability: technical ground truth and carefully designed test cases remain essential.

4.2 First user workflow

- A student uploads permitted notes or selects public material.
- The agent selects a bounded tool sequence to retrieve specific passages.
- The agent drafts a source-grounded explanation and optionally a practice question.
- A verifier checks that citation identifiers exist and that key quoted passages correspond to the material; deeper factual checking is tested separately.
- The agent revises or explicitly states uncertainty when evidence is insufficient.
- The learner sees the response and evidence; the reliability layer records the observable sequence and checks the final artifact.

4.3 The strict separation of roles

Horizon Study does	Horizon Reliability measures
Finds relevant material	Whether retrieval, ranking and citations were correct
Drafts explanations	Unsupported assertions and how they originated

Uses a checking tool	Whether checker signals were accurate and acted on
Revises answers	Whether errors were fixed or simply hidden
Produces the final answer	Final supported-claim quality and verification cost

A study interface with appealing explanations may become a worthwhile consumer product, but that outcome would not by itself prove that our independent reliability system generalizes. If the evaluation method works only for citation checks, we should report that limitation.

4.4 Potential broader applications

After initial experiments: a simple code-repair agent (tests provide clear pass/fail signals), structured document reconciliation (known fields and arithmetic), and a sandbox service-operation agent (independent final database state). We should not expand all of these simultaneously.

5. Measurement: what “precision” will mean

Accuracy and precision have distinct meanings in evaluation. The project should not report a single invented “agent precision” number. Instead, it should specify a task objective, a reference method and a separate set of metrics for agent quality and detector quality.

Metric	Operational meaning	Failure it catches
Final task success	Share of tasks satisfying independent completion criteria	Agent finishes incorrectly despite a plausible trace
Error detection precision	Confirmed consequential errors / all errors flagged as consequential	Too many unnecessary alerts
Error detection recall	Detected consequential errors / all consequential errors in labeled set	Dangerous misses
False-alarm rate	Correct trajectories incorrectly classified as risky	Wasteful verifications or interruptions
Severity agreement	Agreement of score with deterministic/expert reference	Untrustworthy severity estimates
Recovery calibration	Observed recovery frequency for cases predicted to recover	Overconfident interruption decisions
Final impact	Change in final success/quality when a suspect step is corrected	Confusing correlation with causal impact
Verification economics	Token, latency and human-review cost per checked/usable result	Safety improvement that costs too much

5.1 Why final impact is a harder claim

An error at step 3 followed by an incorrect result at step 8 does not prove step 3 caused the bad outcome. When possible, reset the environment and replay the run with step 3 corrected while holding other conditions as stable as practical. Compare final outcomes over repeated trials. When replay is infeasible, report associations with appropriate uncertainty, not causal conclusions.

5.2 Important limits of model-based grading

An independent LLM can gather relevant topic evidence and apply structured rubrics cheaply, but its answer is not equivalent to truth. Judges may share biases or hallucinate. We will store supporting passages, calibrate with human labels and deterministic checks, allow an “uncertain” verdict, and report agreement between human reviewers for ambiguous cases.

6. Research program and experiment design

6.1 Primary hypothesis

A policy using error location, severity, persistence, recovery evidence and verified outcomes can achieve better reliability–cost trade-offs than simple rule-based or full-step checking on a defined family of multi-step tasks.

6.2 Comparators and conditions

Policy	What it checks	Why it matters
No check	Accept task completion without added verifier	Raw resource baseline
Final-only	Verify the final artifact/state once	Common low-cost reference
Full-step	Run intermediate checks at every eligible step	More thorough but expensive baseline
Fixed threshold	Check if a simple score crosses a threshold	Tests whether a complex model is necessary
Trajectory-aware	Check based on estimated error persistence, severity and expected cost	Our experimental candidate

6.3 Fault taxonomy for a first benchmark

- Transient wrong classification: later corrected before final submission.
- Compounding wrong assumption: cascades into multiple downstream decisions.
- Silent artifact defect: answer looks acceptable but stored record differs.

- False-success tool response: API reports success but the state did not change.
- Duplicate effect: operation is unintentionally applied twice.
- Alternative valid path: different intermediate sequence produces a correct final result.

6.4 Experiment sequence

Start with 100–300 reproducible tasks and seed known faults. Evaluate detector precision and recall, severity agreement and false alarms. Expand only after the instrumentation is trustworthy. Then run controlled recovery tests and repeated paired replays. Finally compare verification policies on held-out tasks and test whether performance transfers to a second workflow. Counts are initial design targets, not sample-size claims or promises of statistical significance.

6.5 Reporting discipline

Separate seeded-fault detection from naturally occurring agent mistakes; separate final answer quality from final environment correctness; and show uncertainty intervals alongside averages where sample sizes permit. The experiment is a success only if independent measures support the claim, not because the dashboard looks convincing.

7. Roadmap, gates and operating budget

Period	Deliverable	Gate to proceed
Weeks 1–3	Bounded study agent; file/source retrieval; replaceable model and free mock mode; structured trace capture; deterministic final checks	Reproducible local demo, test coverage and inspected trace
Weeks 4–6	Reference suite, fault injector, trajectory schema, initial error/severity scoring and first benchmark report	Detector comparison and documented failures
Weeks 7–10	Recovery analysis; selective verification compared with final-only, full-step and simple threshold	Measurable reliability/cost difference on held-out tasks
Weeks 11–13	Second-domain test; minimal dashboard; consent-based student/user pilot; external methodological review	Transfer, adoption and credible continue/revise/stop recommendation

7.1 Tools and sources

VS Code, Python, Git/GitHub, SQLite or JSONL, a small test runner and Codex are sufficient for the first loop. A model API or local model can be used behind an adapter. NVIDIA NeMo, OpenTelemetry-compatible export and additional observability software should be added when they produce clear engineering value—not because they are fashionable.

7.2 Cost and resource model

Principal costs: inference tokens and tool calls, storage, document processing, repeated benchmark evaluations, hosting for a beta, optional reviewer time, privacy/security reviews and domain administration. No vendor-specific prices or total budget are claimed here. Before paid integrations, set a monthly cap and meter verification cost per successful final task.

7.3 Roles and collaborators

Product/research coordination; agent/backend engineer; evaluation scientist or technically experienced reviewer; students willing to provide opt-in usability feedback; domain professionals later when rubrics require specialist knowledge. Codex can accelerate implementation but does not replace independent technical review or experimental validation.

8. Business possibilities, risks and responsible deployment

8.1 Two different potential markets

Horizon Reliability, if technically differentiated, may be a developer tool or service sold to teams deploying AI agents with meaningful error costs. Horizon Study, if it earns repeated student usage, may develop into an education product. These could eventually be separate products. We should not imply education subscriptions automatically finance agent-reliability research or that a reliability dashboard already has paying buyers.

8.2 Existing work and competition

Tracing, evaluation and agent observation are established categories. The research brief identifies AgentBoard (step progress), STeCa (step-level calibration), EarlyEval (early-outcome prediction), SCOUT (post-execution probing), NVIDIA NeMo (workflow observation/evaluation), and commercial agent-monitoring platforms as relevant precedents. Our proposed combination is not automatically novel: novelty and usefulness must be shown through direct comparisons and buyer interviews.

8.3 Risks

Risk	Mitigation and decision criterion
Existing methods already solve enough	Compare against real baselines; stop or narrow the

	claim if the incremental value is too small
Agent behaviour is unobservable internally	Study tool calls, state changes and artifacts without requiring hidden chain-of-thought
“Correct” is ambiguous	Use deterministic tasks first; expert disagreement and uncertainty later
High monitoring costs	Publish marginal cost vs avoided misses, including false alarms
No buyer demand	Interview developers and students separately before building advanced product features
Student data and minors	Use public/synthetic materials initially; consent, minimization and retention limits before pilot
Overreach into regulated decisions	No unreviewed real-world legal/medical actions; sandbox by default

8.4 Intellectual honesty

The proposed system has not yet been shown to improve agent reliability, cost, student learning or commercial outcomes. All example trajectories and scores in presentations are illustrative. The goal of the first stage is to discover what can be measured, what can be predicted and what cannot be justified.

9. Immediate actions and collaborator briefing

A practical first 10-day sequence

- Create a clean repository, README, environment setup, issue tracker and an inexpensive model adapter with mock mode.
- Implement one narrow study task: answer questions from a permitted source and expose exact supporting passages.
- Record run-level and step-level structured events; capture the final artifact.
- Write a deterministic citation/reference checker and seed examples with deliberately wrong citations or passages.
- Produce a simple report: where the first defect occurred, the supporting evidence, whether the agent revised and whether the final response passed.
- Run small repeated trials and publish both successful and failed examples internally before expanding scope.

What we can appropriately ask potential collaborators

Technical help to design or critique trace schemas and counterfactual experiments; feedback on realistic agent errors and current observability tools; assistance creating trustworthy evaluation

rubrics; introductions to developers who deploy tool-using agents; controlled user testing of Horizon Study.

What we should not claim yet

Do not advertise universal precision tracking, guaranteed error prediction, demonstrated cost savings, access to proprietary NVIDIA models as a competitive moat, proven demand, or an original technique before experiments and prior-art review support those statements.

Proposed 30-second external description

“Project Horizon is exploring a software layer that measures how errors emerge, propagate and recover during multi-step AI-agent tasks, and whether those errors materially affect real outcomes. Our goal is to make agent verification more selective and cost-effective. We are building a small, useful study agent as our first controlled test environment while developing and benchmarking the reusable reliability engine.”

Selected learning and technical references

Yao et al., ReAct: https://arxiv.org/abs/2210.03629

Anthropic, Building Effective Agents: https://www.anthropic.com/engineering/building-effective-agents

Anthropic, Demystifying Evals: https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

NVIDIA, NeMo Agent Toolkit: https://github.com/NVIDIA/NeMo-Agent-Toolkit

NVIDIA, Evaluation: https://docs.nvidia.com/nemo/agent-toolkit/latest/improve-workflows/evaluate.html

NVIDIA, Observability: https://docs.nvidia.com/nemo/agent-toolkit/latest/run-workflows/observe/observe.html

Background literature reviewed in the Project Horizon AI Agents, Error Trajectories, and Proof-of-Effect research map (October 2026).